

Universidad de los Andes

ISIS-1611: Inteligencia Artificial

Taller 1: Búsqueda con y sin información

Grupo 3

Isabela Archbold (202420166), Isabella Largacha (202420593), Juan David Forero (202411707), Samuel Zuluaga (202520012) & Juan José Castro (202520030)

27 de agosto de 2026

Contenido

Contenido.....	2
1. Introducción y ejecución.....	3
2. Punto 1 — Búsqueda en profundidad (DFS)	3
3. Punto 2 — Búsqueda en amplitud (BFS)	3
4. Punto 3 — Costo uniforme (UCS)	3
5. Punto 4 — Módulo obligatorio y costo dependiente del estado.....	3
6. Punto 5 — A* y diseño de heurísticas.....	4
6.1 Implementación de aStarSearch	4
6.2 manhattanHeuristic y euclideanHeuristic	4
6.3 systemRepairHeuristic.....	4
6.4 Resultados comparativos.....	4
7. Punto 6a — Análisis de los algoritmos y heurísticas.....	6
7.1 Complejidad temporal y espacial.....	6
7.2 Completitud.....	6
7.3 Optimalidad.....	6
7.4 Por qué hasModule debe formar parte del estado	6
7.5 Consistencia de las heurísticas.....	6
7.6 Comparación de las cuatro heurísticas.....	6
8. Punto 6b — Reflexión sobre el uso de inteligencia artificial	6
Anexo A — Tabla completa de resultados del punto 5.....	8
Anexo B — Registro de uso de inteligencia artificial	11
depthFirstSearch	11
breadthFirstSearch	11
uniformCostSearch	11
ModuleRepairProblem.....	11
aStarSearch.....	11
manhattanHeuristic	12
euclideanHeuristic	12
systemRepairHeuristic.....	12

1. Introducción

Este taller aborda tres misiones de búsqueda: diagnóstico, cuyo objetivo es llegar al terminal D; reparación de módulo, que requiere recoger M y finalizar en C; y reparación de sistemas, que consiste en recoger K, reparar todos los T y regresar a C. Para la misión de diagnóstico, se ejecutan DFS, BFS y UCS sobre los mismos mapas, con el fin de comparar su desempeño.

Se implementó los algoritmos DFS, BFS, UCS y A* en search.py, completó los tres métodos solicitados en problems.py y desarrolló las heurísticas en heuristics.py. El archivo agents.py integra los algoritmos con los problemas y permite registrar las métricas utilizadas en la comparación: tiempo de búsqueda, costo del camino y número de nodos expandidos.

2. Punto 1 - Búsqueda en profundidad (DFS)

DFS utiliza una pila (LIFO) para la frontera de búsqueda. Cada elemento de la pila es una tupla de estados y acciones, donde las acciones son el camino acumulado desde el inicio hasta ese estado. Al encontrar la meta retorna el camino recopilado hasta ese punto. Se implemento como Graph Search y no como Tree Search. Adicionalmente se utiliza un set de estados visitados, donde se añaden todos los nodos extraídos. Esto garantiza no caer en ciclos de búsqueda.

Mapas Evaluados:

TinyDiagnostic:

```
RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 95 in 0.0 seconds
Search nodes expanded: 66
¡Misión completada! Costo: 95
```

ServiceDeckTerminal:

```
RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(21, 11)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 212 in 0.0 seconds
Search nodes expanded: 174
¡Misión completada! Costo: 212
```

RadiationShorcut:

```
RobotPositions: [(2, 3)]
MissionSymbols: {'D': [(24, 13)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 172 in 0.0 seconds
Search nodes expanded: 180
¡Misión completada! Costo: 172
```

CoolantBypass:

```
RobotPositions: [(4, 13)]
MissionSymbols: {'D': [(24, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 87 in 0.0 seconds
Search nodes expanded: 65
¡Misión completada! Costo: 87
```

ResearchAtrium:

```

RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(27, 14)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 263 in 0.0 seconds
Search nodes expanded: 250
¡Misión completada! Costo: 263

```

cargoDeckBypass:

```

RobotPositions: [(5, 14)]
MissionSymbols: {'D': [(29, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 117 in 0.0 seconds
Search nodes expanded: 125
¡Misión completada! Costo: 117

```

ReactorGate:

```

RobotPositions: [(3, 17)]
MissionSymbols: {'D': [(31, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 250 in 0.0 seconds
Search nodes expanded: 215
¡Misión completada! Costo: 250

```

AnntenaServiceRing:

```

RobotPositions: [(4, 3)]
MissionSymbols: {'D': [(32, 17)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 326 in 0.0 seconds
Search nodes expanded: 390
¡Misión completada! Costo: 326

```

SolarWingDiagnostic:

```

RobotPositions: [(3, 10)]
MissionSymbols: {'D': [(35, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 300 in 0.0 seconds
Search nodes expanded: 278
¡Misión completada! Costo: 300

```

AnntenaServiceRing:

```

RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 196 in 0.0 seconds
Search nodes expanded: 192
¡Misión completada! Costo: 196

```

En los distintos mapas de prueba, el algoritmo encuentra una solución. Sin embargo, al no planear su recorrido basado en los costos de cada estado, el costo total de la búsqueda no es óptimo. Al correr la interfaz gráfica es posible observar como el robot tiende a dar más pasos de los necesarios para llegar al objetivo, aumentando el costo total de búsqueda y demostrando que no es una solución óptima comparado con otros algoritmos de búsqueda como UCS.

3. Punto 2 - Búsqueda en amplitud (BFS)

BFS utiliza una cola (FIFO) para la frontera de búsqueda. Cada elemento de la cola es una tupla de estado y acciones, donde las acciones representan el camino acumulado desde el inicio hasta ese estado. Al encontrar la meta, retorna el camino recopilado hasta ese punto. Se implementó como Graph Search y no como Tree Search. Adicionalmente, se utiliza un set de estados visitados, al cual se añaden los nodos cuando ingresan a la cola. Esto evita añadir repetidamente el mismo estado, garantiza no caer en ciclos y permite explorar los estados por niveles de profundidad. Por ello, BFS encuentra el camino con la menor cantidad de acciones, siempre que todas tengan el mismo costo.

Mapas evaluados:

tinyDiagnostic:

```
RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 147
¡Misión completada! Costo: 24
```

serviceDeskTerminal:

```
RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(21, 11)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 26 in 0.0 seconds
Search nodes expanded: 170
¡Misión completada! Costo: 26
```

radiationShortcut:

```
RobotPositions: [(2, 3)]
MissionSymbols: {'D': [(24, 13)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 84 in 0.0 seconds
Search nodes expanded: 244
¡Misión completada! Costo: 84
```

coolantBypass:

```
RobotPositions: [(4, 13)]
MissionSymbols: {'D': [(24, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 30 in 0.0 seconds
Search nodes expanded: 236
¡Misión completada! Costo: 30
```

researchAtrium:

```
RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(27, 14)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 70 in 0.0 seconds
Search nodes expanded: 312
¡Misión completada! Costo: 70
```

cargoDeckBypass:

```
RobotPositions: [(5, 14)]
MissionSymbols: {'D': [(29, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 71 in 0.0 seconds
Search nodes expanded: 320
¡Misión completada! Costo: 71
```

reactorGate:

```
RobotPositions: [(3, 17)]
MissionSymbols: {'D': [(31, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 108 in 0.0 seconds
Search nodes expanded: 366
¡Misión completada! Costo: 108
```

antennaServiceRing:

```
RobotPositions: [(4, 3)]
MissionSymbols: {'D': [(32, 17)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 86 in 0.0 seconds
Search nodes expanded: 410
¡Misión completada! Costo: 86
```

solarWingDiagnostic:

```
RobotPositions: [(3, 10)]
MissionSymbols: {'D': [(35, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 77 in 0.0 seconds
Search nodes expanded: 376
¡Misión completada! Costo: 77
```

orbitalDiagnosticComplex:

```
RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 101 in 0.0 seconds
Search nodes expanded: 591
¡Misión completada! Costo: 101
```

4. Punto 3 - Costo uniforme (UCS)

UCS es exactamente el mismo algoritmo que se usaba en EDA; Dijkstra. A diferencia de los otros algoritmos, como BFS o DFS, que uno usa una pila y el otro una cola, este utiliza una cola de prioridad orientada a menor. La cola de prioridad analiza el costo acumulado de la expansión y ordena acorde al algoritmo. Al ser una cola de prioridad extra el primer elemento (el más barato), verificando si es la meta, y si no expande sus sucesores insertando en la cola de prioridad los nuevos costos acumulados.

Resultados:

tinyDiagnostic:

```
RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 134
¡Misión completada! Costo: 24
```

ServiceDeckTerminal:

```
RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(21, 11)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 26 in 0.0 seconds
Search nodes expanded: 155
¡Misión completada! Costo: 26
```

RadiationShortcut:

```
RobotPositions: [(2, 3)]
MissionSymbols: {'D': [(24, 13)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 51 in 0.0 seconds
Search nodes expanded: 251
¡Misión completada! Costo: 51
```

CoolantBypass:

```
RobotPositions: [(4, 13)]
MissionSymbols: {'D': [(24, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 30 in 0.0 seconds
Search nodes expanded: 196
¡Misión completada! Costo: 30
```

ResearchAtrium:

```
RobotPositions: [(3, 3)]
MissionSymbols: {'D': [(27, 14)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 49 in 0.0 seconds
Search nodes expanded: 307
¡Misión completada! Costo: 49
```

CargoDeckBypass:

```
RobotPositions: [(5, 14)]
MissionSymbols: {'D': [(29, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 58 in 0.0 seconds
Search nodes expanded: 339
¡Misión completada! Costo: 58
```

ReactorGate:

```
RobotPositions: [(3, 17)]
MissionSymbols: {'D': [(31, 3)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 55 in 0.0 seconds
Search nodes expanded: 346
¡Misión completada! Costo: 55
```

AntennaServiceRing:

```
RobotPositions: [(4, 3)]
MissionSymbols: {'D': [(32, 17)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 59 in 0.0 seconds
Search nodes expanded: 413
¡Misión completada! Costo: 59
```

SolarWingDiagnostic:


```

RobotPositions: [(3, 10)]
MissionSymbols: {'D': [(35, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 59 in 0.0 seconds
Search nodes expanded: 437
¡Misión completada! Costo: 59

```

OrbitalDiagnosticComplex:

```

RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 62 in 0.0 seconds
Search nodes expanded: 560
¡Misión completada! Costo: 62

```

UCS usa el costo acumulado como prioridad porque los movimientos en este taller no tienen costo uniforme debido a todos los tipos de terreno presentes en los mapas, por ende, a diferencia de BFS si tiene en cuenta atravesar terrenos deteriorados que consuman más energía y por ende más costo para el recorrido. Aunque BFS en algunos casos tiene los mismos movimientos que UCS, UCS usa un camino más barato.

5. Punto 4 - Módulo obligatorio y costo dependiente del estado

Para esto se reutilizó uniformCostSearch implementado en el punto 3 y se completó la clase ModuleRepairProblem. El estado se representa mediante la tupla (position, hasModule), donde position corresponde a la ubicación actual del robot y hasModule indica si ya recogió el módulo de energía M. La función isGoalState determina que la misión termina únicamente cuando el robot se encuentra en el centro de control C y hasModule es verdadero. La función getSuccessors genera movimientos hacia el norte, sur, este y oeste, saca los movimientos que conducen a muros y retorna cada sucesor en el formato nextState, direction, stepCost. Cuando el robot entra en M, el valor de hasModule del siguiente estado cambia a verdadero. La función _getStepCost obtiene el costo normal del terreno de destino. Antes de recoger M cada movimiento conserva este costo después de recogerlo los movimientos cuestan el doble. El paso utilizado para entrar en M todavía tiene costo normal porque stepCost se calcula utilizando el valor de hasModule anterior a la transición. El valor se actualiza solamente después de calcular el costo. La implementación se ejecutó en los diez mapas de layouts module. Todas las finalizaron correctamente.

Mapa	Costo	Nodos expandidos
cargoLoopDeck	136	285
controlWingCargo	74	290

floodedServiceDeck	99	372
loadingSpine	123	632
moduleRelayRooms	159	525
moduleVault	80	245
orbitalCargoMaze	167	797
reactorSupplyBay	85	310
stationModuleComplex	208	1122
tinyModuleRun	48	138

Estos resultados muestran que UCS encontró una solución válida en todos los mapas respetando la submeta obligatoria de recoger M antes de terminar en C y aplicando el costo dependiente del estado.

6. Punto 5 - A* y diseño de heurísticas

En este punto se implementó el algoritmo A* y se diseñaron tres heurísticas para resolver la misión de recoger el kit K, reparar todos los sistemas T y finalizar en el centro de control C. Se compararon las heurísticas Manhattan, Euclidean y una heurística propia, junto con nullHeuristic, evaluando el costo de la solución, los nodos expandidos y el tiempo de búsqueda.

Tabla 1. Nodos expandidos por heurística sobre mapas representativos.

Mapa	T	Costo	Null	Manhattan	Euclidean	Propia
tinyRepair	1	24	148	127	129	35
Evidencia P5-01 — tinyRepair. <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i tinyRepair -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 24 in 0.0 seconds Search nodes expanded: 148 Misión completada! Costo: 24</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i tinyRepair -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 24 in 0.0 seconds Search nodes expanded: 148 Misión completada! Costo: 24</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i tinyRepair -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 24 in 0.0 seconds Search nodes expanded: 148 Misión completada! Costo: 24</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i tinyRepair -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 24 in 0.0 seconds Search nodes expanded: 148 Misión completada! Costo: 24</pre>						
dualArray	2	50	284	263	263	135
Evidencia P5-02 — dualArray. <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i dualArray -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1], [1, 5]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 50 in 0.0 seconds Search nodes expanded: 284 Misión completada! Costo: 50</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i dualArray -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1], [1, 5]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 50 in 0.0 seconds Search nodes expanded: 284 Misión completada! Costo: 50</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i dualArray -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1], [1, 5]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 50 in 0.0 seconds Search nodes expanded: 284 Misión completada! Costo: 50</pre> <pre># LaboSearchOnMapasMapas-Machon-Air-5 EstacionEspacial % python main.py -p SystemRepairProblem -f aStarSearch -h nullHeuristic -i dualArray -q # SuboPositions: [12, 7] # MissionSymbols: ['P', [1, 'C', [19, 7]], 'M', [1, 'K', [15, 3]], 'T', [113, 1], [1, 5]] # SearchAgent using function aStarSearch and heuristic nullHeuristic # SearchAgent using problem type SystemRepairProblem Misión: Recoger K, reparar todos los T y volver a C Tipo de misión: system Path found with total cost of 50 in 0.0 seconds Search nodes expanded: 284 Misión completada! Costo: 50</pre>						
controlWing	3	69	781	668	678	191

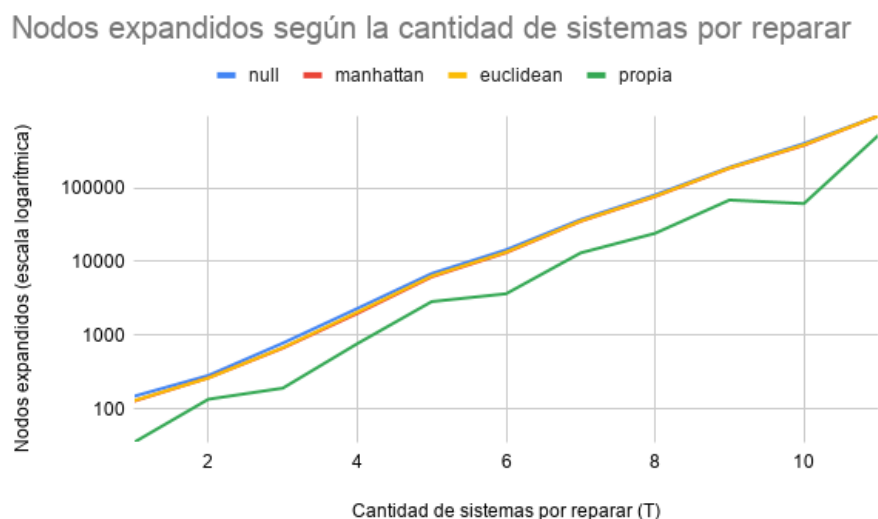
Nota: El costo de la solución es idéntico para las cuatro heurísticas en todos los mapas.

Mapa	T	null	manhattan	euclidean	propia
fourCornerArray	4	0.0	0.0	0.0	0.0
sevenNodeRing	7	0.1	0.2	0.2	0.1
sensorArray	8	0.3	0.4	0.5	0.2
nineBlockGrid	9	0.8	1	1.2	0.5
repairCluster	10	2.1	2.4	2.8	0.5
spiralWing	10	2.3	3	3.3	1.8
outerRing	11	6	7.8	8.4	1.9
elevenRoomDeck	11	5.6	6.8	7.5	4.8

Tabla 3. Instancias grandes resueltas únicamente con la heurística propia.

Mapa	T	Costo	Nodos expandidos	Tiempo (s)
balancedArray	13	216	792.117	8.7
spreadStation	13	248	2.269.604	29
longServiceWing	14	208	2.782.295	39.4
reactorMaze	14	254	4.242.533	68.8
fifteenOpenDeck	15	267	12.457.758	1784.5

Figure 1. Nodos expandidos por A* según la cantidad de sistemas T.



Nota. Hecha en Google Sheets.

En general, los nodos expandidos aumentan al incrementar la cantidad de sistemas por reparar. Sin embargo, la dificultad también depende de la distribución de los objetivos y de los obstáculos, por lo que dos mapas con cantidades similares de T pueden tener resultados diferentes. La heurística propia expande menos nodos que las otras tres en todos los casos representados.

7. Punto 6a — Análisis de los algoritmos y heurísticas

7.1 Complejidad temporal y espacial

DFS

Teniendo en cuenta que el factor máximo de ramificación b es igual a 4 (norte, sur, este, oeste) y que la profundidad m máxima es la cantidad de celdas (el robot visita todas las celdas)

Complejidad temporal: $O(b^m)$

Complejidad espacial (m)

Este algoritmo de búsqueda es completo, pero no es óptimo. Encuentra una solución válida pero no está garantizado que sea la más óptima. DFS encuentra la mejor solución en donde la solución óptima se encuentre a profundidad en el árbol de decisión y es especialmente útil cuando el espacio en memoria es muy limitado, ya que nunca guarda más de la máxima profundidad del árbol. Asimismo, al no tener en cuenta los costos de cada estado, dfs resulta siendo mucho más costoso (óptimo) que otras soluciones como UCS o A*, las cuales siempre devolverán un camino verificando la mayor optimalidad posible.

BFS

Teniendo en cuenta que el factor máximo de ramificación (b) es igual a 4 (norte, sur, este y oeste) y que d representa la profundidad de la solución más cercana. La complejidad temporal $O(b^d)$ y la complejidad espacial $O(b^d)$. Este algoritmo de búsqueda es completo porque garantiza encontrar una solución si existe. Es óptimo cuando las acciones tienen el mismo costo, ya que encuentra el camino con la menor cantidad de movimientos. En el caso de que las acciones tengan costos diferentes, BFS solo considera el camino con menor cantidad de saltos, no de costo porque no utiliza stepCost para decidir que nodo explorar. Una desventaja de BFS es su elevado costo de memoria porque puede llegar a almacenar a todos los nodos de un mismo nivel.

UCS

Teniendo en cuenta que el factor máximo de ramificación b es igual a 4 (norte, sur, este, oeste) y que el costo de la solución óptima es C^* con un costo mínimo por paso ϵ (en este taller, $\epsilon = 1$, el costo del corredor normal)

Complejidad temporal: $O(b^{(1+C^*/\epsilon)})$

Complejidad espacial: $O(b^{(1+C^*/\epsilon)})$

A*

En el peor caso, A* tiene complejidad temporal y espacial exponencial, $O(b^d)$. Sin embargo, su desempeño depende de la calidad de la heurística; es decir, cuanto más informativa sea, menos nodos expande.

7.2 Completitud

DFS

DFS es completo ya que siempre, ya que el espacio de estados es finito y existe un manejo para evitar ciclos. Entonces es posible encontrar una solución siempre. Sin embargo, esta solución no tiene en cuenta los costos de la ruta encontrada.

BFS

BFS es completo porque el espacio de estados es finito y utiliza un conjunto de estados visitados para evitar ciclos. Si existe una solución, BFS garantiza encontrarla. Sin embargo, no tiene en cuenta los costos de las acciones, por lo que solo garantiza la solución de menor costo cuando todas las acciones tienen el mismo costo.

UCS

UCS es completo ya que siempre encuentra una solución, el único problema es cómo se establecen los costos, si hay costos negativos podrían acumularse pasos infinitos antes de llegar a la meta.

A*

A* es completo en los problemas del taller porque el espacio de estados es finito. Por tanto, si existe una solución, el algoritmo eventualmente la encontrará.

7.3 Optimalidad

DFS

DFS no es óptimo, ya que al no tener en cuenta los costos de los caminos entre estados, no es seguro que el camino entregado sea el menos costoso. A diferencia de otros algoritmos de búsqueda, DFS no tiene un control de pesos acumulados.

BFS

BFS es óptimo cuando todas las acciones tienen el mismo costo, ya que explora los estados por niveles y encuentra primero el camino con la menor cantidad de acciones. Sin embargo, si las acciones tienen costos diferentes, BFS no garantiza que el camino encontrado sea el menos costoso, porque no lleva un control de los costos acumulados.

UCS

UCS es óptimo porque siempre encuentra una solución siguiendo su lógica, buscando el menor costo acumulado. Siempre y cuando todos los costos no sean negativos. En este caso, en todos los mapas el costo mínimo era de 1, por lo cual UCS fue óptimo.

A*

A* es óptimo cuando la heurística nunca sobreestima el costo real restante. Además, como la implementación cierra cada estado la primera vez que lo extrae de la frontera y no permite reaperturas, la heurística también debe ser consistente. Bajo estas condiciones, A* encuentra una solución de costo mínimo.

7.4 Por qué hasModule debe formar parte del estado

La posición del robot no es suficiente para representar este problema porque el robot puede encontrarse en una misma celda en dos situaciones diferentes, antes y después de recoger el módulo. Por ejemplo, los estados $((x, y), \text{False})$ y $((x, y), \text{True})$ tienen la misma posición, pero no son iguales. En el primero llegar a C no completa la misión y los movimientos todavía tienen su costo normal mientras que, en el segundo, llegar a C sí puede cumplir con la meta y cada movimiento posterior cuesta el doble. Si el estado estuviera representado solamente por la posición el conjunto de visitados de UCS podría marcar una celda antes de recoger M y sacar luego la misma celda cuando el robot ya lleva el módulo. Esto eliminaría rutas válidas y podría impedir que la búsqueda encuentre la solución óptima o incluso cualquier solución. Por esta razón, hasModule debe formar parte del estado.

7.5 Consistencia de las heurísticas

En estos mapas, cada movimiento válido tiene costo 1. Mientras el objetivo de referencia no cambia, tanto Manhattan como Euclidean cumplen la desigualdad triangular: moverse a una celda vecina puede reducir la distancia estimada como máximo en una unidad. Cuando quedan varios sistemas pendientes, tomar la distancia mínima entre ellos conserva esta propiedad mientras el conjunto no cambie.

También deben considerarse las transiciones entre fases. Justo antes de recoger el kit, la distancia hasta K es 1; después de recogerlo, la nueva estimación es no negativa. Por tanto, se cumple $h(s) \leq 1 + h(s')$. Lo mismo sucede al entrar en un sistema pendiente y repararlo: antes de ese movimiento, la distancia al sistema pendiente más cercano es como máximo 1, y la estimación posterior es no negativa, aunque cambie el objetivo.

Finalmente, cuando no quedan sistemas pendientes, ambas heurísticas estiman la distancia hasta C y retornan 0 al llegar a la meta. En consecuencia, cumplen la condición de consistencia en las transiciones del problema.

7.6 Comparación de las cuatro heurísticas

Se evidencia, por ejemplo, que en las cuatro ejecuciones verificadas de fourCornerArray, A* completó la misión con costo 78. Sin embargo, el esfuerzo de búsqueda fue diferente: nullHeuristic expandió 2305 nodos, Manhattan 1972, Euclidean 2053 y la heurística propia 767. Por tanto, la propia redujo aproximadamente un 66,7 % las expansiones respecto a nullHeuristic, manteniendo el mismo costo de solución.

Esta diferencia se explica por la información utilizada. Manhattan y Euclidean estiman únicamente la distancia al siguiente objetivo de referencia, mientras que la heurística propia incorpora las distancias del laberinto y el regreso al centro de control mediante cotas que consideran los sistemas pendientes.

Las cuatro ejecuciones mostraron un tiempo de 0.0 segundos con la precisión de salida del programa. Este resultado no significa que la búsqueda fuera instantánea ni permite establecer cuál fue más rápida en este mapa. Para comparar tiempos deben utilizarse instancias mayores o mediciones con mayor precisión.

La coincidencia de costos es compatible con la optimalidad esperada de A* bajo sus condiciones teóricas, pero no constituye por sí sola una demostración de admisibilidad o consistencia; estas propiedades requieren los argumentos anteriores.

8. Punto 6b — Reflexión sobre el uso de inteligencia artificial

DFS

Antes de empezar a escribir código, decidí recurrir a Claude para decidir las distintas opciones del planteamiento del algoritmo. Gracias a Estructuras de datos y Algoritmos, ya tenía cierto conocimiento en recorridos sobre grafos, pero quería saber si existían otras documentaciones que me permitieran realizar la función. Para esto, el prompt utilizado fue el primero en el anexo B, en la parte de DFS. Claude proporciono distintos enlaces de documentación de resolución de búsqueda implementando dfs y adicionalmente añadió que la mejor opción era utilizar un recorrido de grafo, ya que usualmente grilla se modela con este (ver Anexo B DFS).

Partiendo de esto empecé a realizar la función DFS que fuera funcional para el contexto del taller. Utilicé las documentaciones sugeridas por Claude para saber el orden que debe seguir el algoritmo y también me basé en la función de búsqueda de grafo creada por mí en el curso de Estructuras de datos y Algoritmos. Al intentar probarlo, no funcionó y recurrí después de varios intentos fallidos, recurrí de nuevo a Claude para lograr entender que parte del algoritmo estaba realizando erróneamente. Claude fue capaz de explicarme que parte del pseudocódigo de las documentaciones no estaba entendiendo, expresó lo siguiente: " Aquí cada elemento

de la pila guarda solo una acción individual (action), no el historial acumulado como debería ser. Luego, cada vez que haces pop, agregas esa acción a una lista compartida actions con .append(). El problema: en DFS con pila, cuando el algoritmo "retrocede" (backtracking) — es decir, cuando agota un camino sin éxito y saca de la pila un estado que pertenece a una rama completamente distinta — tu lista actions no se limpia ni se corrige. Sigue creciendo con acciones de ramas abandonadas que ya no llevan a ningún lado".

Aprendizaje:

Este ejercicio me permitió confirmar que el planteamiento teórico de un algoritmo puede tener distintos modos de implementación, los cuales varían dependiendo del contexto y el ambiente en el cual el agente se desarrolle. Gracias a la explicación dada por la IA, entendí que la solución a los ciclos infinitos no estaba en la lógica del código, sino en la estructura que utilizara para la búsqueda. Adicionalmente, pude comprender errores que cometí dentro de la construcción de la función al guardar las acciones individualmente en vez del camino completo, lo que me permitió entender mejor el algoritmo de búsqueda y posteriormente corregirlo para que fuera funcional. Los prompts utilizados se encuentran en el anexo B.

BFS

Antes de realizar mi parte recurrí a los trabajos realizados en el curso de Estructuras de Datos y Algoritmos. Con conocimiento de grafos adquirido, sabía que podría desarrollar el algoritmo, pero aun así decidí consultar con inteligencia artificial para ver si existía una alternativa más efectiva. Para esto utilicé el prompt "Quiero saber si mi implementación de BFS es efectiva" con la implementación del algoritmo adjuntada en el mismo chat. La inteligencia artificial no hizo mayores cambios en el algoritmo, por lo cual implementé una versión muy parecida a la versión inicial. El tema en el que más me ayudó la inteligencia artificial fue entendiendo la manera en la que se tenían que llevar a cabo las implementaciones. Existían muchas variables y nombres que no coincidían con la implementación inicial creada en la clase de Estructuras de Datos y Algoritmos. Gracias a ChatGPT entendí cuál era el significado de cada término para poder transformar mi implementación pasada a algo que funcionara en el proyecto actual.

UCS

Antes de empezar a escribir código, decidí recurrir a Claude para entender cómo trasladar mi conocimiento previo de Dijkstra implementado en el curso de Estructuras de Datos y Algoritmos con mis propias estructuras de mapa, cola de prioridad y pila hacia la interfaz específica de este proyecto, que ya trae sus propias abstracciones en algorithms/utils.py.

Para esto, el prompt utilizado enunciaba lo siguiente: "Tengo mi propia implementación de Dijkstra hecha en Estructuras de Datos, con mis propias estructuras de mapa, cola de prioridad y pila para reconstruir el camino. Sé que UCS es básicamente lo mismo, pero este proyecto ya tiene sus propias estructuras en utils.py. ¿Me puedes armar una tabla comparando qué usé yo en mi Dijkstra contra qué debería usar aquí, y darme pseudocódigo genérico, no el código de la función?" Claude respondió con una tabla de equivalencias y, entre otras cosas, añadió lo siguiente: "No existe grafo precomputado: llamas problem.getSuccessors(state), que te devuelve directo una lista de tuplas (vecino, acción, costo)... En vez de guardar predecesores y reconstruir con un stack (path_to), cada elemento que metes a la cola ya lleva pegada la lista de acciones acumuladas hasta ese estado" (Claude, Anthropic, 2026).

Partiendo de esto, escribí mi propia función reusando la lógica de mi Dijkstra de EDA, adaptando las estructuras según la tabla de equivalencias. Al probarla, no detectaba la meta y la búsqueda no terminaba correctamente. Le mostré el código a Claude para que identificara el error, sin pedirle que lo corrigiera. Claude señaló lo siguiente: "la condición `if estadoActual is problem.isGoalState(estadoActual)`: compara por identidad de objeto en vez de evaluar el booleano retornado, lo que impide detectar la meta. También el `utils.raiseNotDefined()` residual al final debería ser `return []`, siguiendo el patrón ya usado en DFS y BFS" (Claude, Anthropic, 2026).

Aprendizaje: Este ejercicio me confirmó que "traducir" un algoritmo de una implementación con estructuras propias a un framework ajeno no es reescribirlo desde cero, sino identificar la equivalencia funcional entre estructuras — mi cola de prioridad de EDA con la de este proyecto, y mi reconstrucción de camino con predecesores contra acumular acciones directamente en cada nodo de la frontera. El error que cometí (is en vez de evaluar el booleano) fue un error sintáctico básico, no conceptual: el diseño del algoritmo — cola de prioridad, marcar visitados al expandir, verificar meta al sacar de la cola — lo tenía correcto desde el primer intento, apoyado en mi conocimiento previo de Dijkstra. Verifiqué el resultado corriendo `uniformCostSearch` contra los 10 mapas de `layouts/diagnostic`, comparando el costo obtenido contra `breadthFirstSearch`: en todos los casos el costo de UCS fue igual o menor, nunca mayor, confirmando que la corrección respeta la optimalidad esperada del algoritmo.

A*

El apoyo de Claude me permitió aclarar conceptos de A* y revisar la presentación de los resultados. Considero importante utilizar la IA como una herramienta de aprendizaje y no delegarle todo el desarrollo del taller. Para aprovecharla mejor, las preguntas deben enfocarse en entender conceptos e identificar errores. Por ejemplo, pregunté ¿No sería mejor hacer un anexo?, para organizar las capturas sin sobrecargar las tablas; y ¿Qué más me falta por llenar de la guía?, para revisar los requisitos pendientes.

Compartir las salidas de terminal de `fourCornerArray` y la ejecución demorada de `fifteenOpenDeck` permitió distinguir entre resultados verificados y datos pendientes de comprobación. También comprendí que un tiempo mostrado como 0.0 segundos no significa que la búsqueda sea instantánea y, por otro lado, que se debe tener paciencia cuando las ejecuciones son muy largas. La IA facilitó la organización de tareas repetitivas, como completar las tablas, pero la responsabilidad de comprobar los datos siguió siendo mía.

Me llevo como aprendizaje que aceptar una respuesta de IA sin poder explicarla limita el valor del ejercicio. Ahora puedo explicar con mayor claridad cómo A* utiliza la prioridad $g(n) + h(n)$, aunque necesito seguir repasando la demostración de consistencia para sustentarla con seguridad.

Modulo

Durante este ejercicio utilicé ChatGPT como acompañamiento para identificar las funciones que debía completar, comprender el estado compuesto y organizar las pruebas. La orientación me permitió comprender por qué la posición del robot no era suficiente y era necesario incluir `hasModule`. También entendí que el costo del movimiento para entrar en M debe calcularse antes de actualizar ese valor, de manera que conserve el costo normal y el costo doble empiece en el movimiento siguiente. Las sugerencias recibidas fueron importantes para organizar la implementación, pero verifiqué personalmente el código mediante `py_compile`, `git diff --check` y la ejecución de los diez mapas de `layouts/module`. Todos finalizaron correctamente. Después de realizar estas pruebas, considero que puedo explicar el funcionamiento de `isGoalState`, `_getStepCost` y `getSuccessors`. El registro detallado de la interacción se encuentra en el Anexo B.

Anexo A — Registro de uso de inteligencia artificial

DepthFirstSearch

- Versión inicial propia (pegar el código, aunque tuviera errores).

```
def depthFirstSearch(problem: SearchProblem):
    frontier = utils.Stack()
    startState = problem.getStartState()
    frontier.push((startState, []))
    visited = set()
    visited.add(startState)
    while not frontier.isEmpty():
        state, actions = frontier.pop()
        if problem.isGoalState(state):
            return actions
        for successor, action, stepCost in problem.getSuccessors(state):
            if successor not in visited:
                visited.add(successor)
                frontier.push((successor, actions + [action]))
    return []
```

- Prompts utilizados:

Quería saber cuál era la mejor variación del algoritmo que me podría servir, por lo que solicite ayuda conceptual y de búsqueda para encontrar documentación que explicara recorridos DFS en grillas:

Necesito realizar un recorrido DFS sobre una grilla para encontrar una solución al problema de búsqueda, ¿de qué forma puedo evitar ciclos y hacerlo de una manera familiar? He trabajado antes con recorridos en árboles y grafos, pero no sé si es mejor realizar recorrido en grafo de estados o en árbol de decisión.

Posteriormente, después de solicitar verificación de la coherencia del código inicial recibí la siguiente sugerencia:

" Aquí cada elemento de la pila guarda solo una acción individual (action), no el historial acumulado como debería ser. Luego, cada vez que haces pop, agregas esa acción a una lista compartida actions con .append(). El problema: en DFS con pila, cuando el algoritmo "retrocede" (backtracking) — es decir, cuando agota un camino sin éxito y saca de la pila un estado que pertenece a una rama completamente distinta — tu lista actions no se limpia ni se corrige. Sigue creciendo con acciones de ramas abandonadas que ya no llevan a ningún lado"

- Sugerencias relevantes recibidas.

Para la metodología de la construcción del recorrido recibí la siguiente sugerencia:

" La grilla se maneja como un grafo, no un árbol. En un árbol, cada nodo tiene un único padre y no hay forma de "volver" a un ancestro por otro camino. En una grilla, cada celda tiene hasta 4 vecinos, y esos vecinos pueden llevarte de vuelta a una celda que ya visitaste (por ejemplo: derecha, abajo, izquierda,

arriba te devuelve al punto de partida). Eso es literalmente un ciclo. Por eso los recorridos de árbol que ya conoces (donde nunca te preocupas por "no volver atrás")

- Cambios aceptados, rechazados o adaptados, con la razón.

Dentro de las ideas de documentación que Claude ofreció, partí por las que especificaban un recorrido no recursivo orientado a graph search, y descarté cualquier otra opción. Empecé a realizar la función DFS que fuera funcional para el contexto del taller. Utilicé las documentaciones sugeridas por Claude para saber el orden que debe seguir el algoritmo y también me basé en la función de búsqueda de grafo creada por mí en el curso de Estructuras de datos y Algoritmos. Al intentar probarlo, no funcionó y recurrí después de varios intentos fallidos, recurrí de nuevo a Claude para lograr entender que parte del algoritmo estaba realizando erróneamente. Claude fue capaz de explicarme que parte del pseudocódigo de las documentaciones no estaba entendiendo

- Cómo se verificó que funciona.

Una vez realizados los cambios, probe nuevamente la función con los casos de búsqueda que ofrece la interfaz. La verificación se realizó comprobando que el robot pudiera encontrar un camino hasta su meta en cada mapa y que las acciones retornadas correspondieran únicamente al camino tomado. Adicionalmente me comprobó que el algoritmo de búsqueda no permite al robot quedarse atrapado en ciclos.

breadthFirstSearch

POR ESCRIBIR — borrar este bloque al terminar

- `def breadthFirstSearch(problem: SearchProblem):`
- `""" Search the shallowest nodes in the search tree first. """`
- `frontier = utils.Queue()`
- `startState = problem.getStartState()`
- `frontier.push((startState, []))`
- `visited = set()`
- `while not frontier.isEmpty():`
- `state, actions = frontier.pop()`
- `if problem.isGoalState(state):`
- `return actions`
- `if state not in visited:`
- `visited.add(state)`
- `for successor, action, stepCost in problem.getSuccessors(state):`
- `if successor not in visited:`
- `frontier.push( (successor, actions + [action]) )`

`return []`

PROMPT:

- "Quiero saber si mi implementacion de BFS es efectiva"
- "Esto es DFS, necesito que implementes BFS con base en este código".
- SUGERENCIAS:
- La IA me explico la diferencia entre DFS y BFS de manera mas directa y con ejemplos. Con esto pude entender mejor la implementacion que tenia antes.


```

        if nuevo_costo < info_w['dist_to']:
            info_w['edge_from'] = vertice_v
            info_w['dist_to'] = nuevo_costo
            estructura['visited'] = mp.put(estructura['visited'], vertice_w, info_w)
            pq.insert(estructura['pq'], nuevo_costo, vertice_w)

    return estructura

```

- Prompts utilizados.
Solicité a Claude una tabla comparativa entre las estructuras de mi implementación de Dijkstra en Estructuras de Datos y Algoritmos (mp, pq, al, st) y las abstracciones equivalentes de este proyecto (utils.PriorityQueue, getSuccessors), sin pedir código de la función, y luego pseudocódigo genérico del algoritmo.
- Sugerencias relevantes recibidas.
Usar las funciones ya creadas en utils, las cuales resumían el código inicial y lo arreglaban.
- Cambios aceptados, rechazados o adaptados, con la razón.
La lógica completa de mi Dijkstra de EDA (cola de prioridad orientada a menor, marcar visitados al extraer, reconstrucción de camino) la adapté a las estructuras de este proyecto siguiendo la tabla de equivalencias que me dio Claude cambié mp/pq/al por utils.PriorityQueue y problem.getSuccessors, y reemplacé la reconstrucción con edge_from + pila por acumular las acciones directamente en cada elemento de la cola.
- Cómo se verificó que funciona.
Se ejecutó uniformCostSearch sobre los 10 mapas de layouts/diagnostic, comparando el costo obtenido contra breadthFirstSearch en los mismos mapas. En todos los casos el costo de UCS fue igual o menor al de BFS, nunca mayor evidencia de que respeta la optimalidad de costo.

ModuleRepairProblem

- Versión inicial propia (pegar el código, aunque tuviera errores).

```

def isGoalState(self, state):
    position, hasModule = state
    if position == self.goalPosition:
        return True
    return False

def _getStepCost(self, nextPosition, hasModule):
    cost = 1
    if nextPosition in self.walls:
        cost = 10
    if hasModule:
        cost = cost + 1
    return cost

def getSuccessors(self, state):

```

```

successors = [ ]
position, hasModule = state
x, y = position
movements = [
    (0, 1),
    (0, -1),
    (1, 0),
    (-1, 0)
]
for dx, dy in movements:
    nextPosition = (x + dx, y + dy)
    if nextPosition not in self.walls:
        nextState = (nextPosition, hasModule)
        cost = self._getStepCost(nextPosition, hasModule)
        successors.append((nextState, (dx, dy), cost))
return successors

```

- Prompts utilizados :

Quiero que me guíes y me expliques paso a paso para poder entender bien el y hacer el trabajo . Durante el proceso también compartí las salidas de py_compile, las ejecuciones de los mapas y git diff --check para solicitar orientación sobre la verificación y los problemas de formato encontrados.

- Sugerencias relevantes recibidas.:

ChatGPT explicó que el estado debía separarse en position y hasModule que exigía estar en self.controlPosition y tener el módulo; y que getSuccessors debía generar los cuatro movimientos cardinales, descartar los muros y conservar el formato (nextState, direction, stepCost). También explicó que stepCost debía calcularse utilizando el valor de hasModule anterior a la transición. De esta manera entrar en M conserva el costo normal y el costo doble se aplica desde el movimiento siguiente.

- Cambios aceptados, rechazados o adaptados, con la razón.:

Acepté las explicaciones sobre el estado compuesto, la condición de meta, el cálculo del costo y la generación de sucesores. Rechacé que la IA implementara directamente toda la solución y pedí que me guiara paso a paso. Escribí personalmente el código, corregí los espacios detectados por git diff --check y verifiqué la implementación ejecutándola en los diez mapas de layouts module.

- Cómo se verificó que funciona:

Primero se comprobó la sintaxis con

```
python3 -m py_compile algorithms/problems.py
```

Después se ejecutó ModuleRepairProblem con uniformCostSearch sobre los diez mapas de layouts/module. Todas las ejecuciones terminaron con el mensaje ¡Misión completada!. En moduleVault se obtuvo un costo de 80 y se expandieron 245 nodos.

Finalmente se ejecutó git diff --check hasta eliminar todas las advertencias de formato y se utilizó git status para comprobar que el único archivo de código modificado fuera algorithms/problems.py.

Versión inicial propia reconstruida

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    frontera = utils.PriorityQueue()
    estadoInicial = problem.getStartState()
    frontera.push(
        (estadoInicial, [], 0),
        heuristic(estadoInicial, problem))
    visitados = set()

    while not frontera.isEmpty():
        estadoActual, acciones, costoAcumulado = frontera.pop()

        if problem.isGoalState(estadoActual):
            return acciones

        if estadoActual not in visitados:
            visitados.add(estadoActual)

        for sucesor, accion, costo in problem.getSuccessors(estadoActual):
            if sucesor not in visitados:
                nuevoCosto = costoAcumulado + costo

                # Error: solo usa la heurística como prioridad
                prioridad = heuristic(sucesor, problem)

                frontera.push(
                    (sucesor, acciones + [accion], nuevoCosto),
                    prioridad
                )

    return []
```

Prompts utilizados

- Estoy implementando A* a partir de uniformCostSearch. ¿Qué debo modificar para incorporar la heurística?
- ¿La prioridad de A* debe ser solamente la heurística o también debe incluir el costo acumulado?
- Revisa si estoy almacenando correctamente las acciones y el costo del camino.

Sugerencias relevantes recibidas

Se indicó que utilizar solamente $h(n)$ como prioridad convierte el algoritmo en una búsqueda voraz. En A*, la prioridad debe calcularse como $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado y $h(n)$ es la estimación del costo restante.

Cambios aceptados, rechazados o adaptados

Acepté cambiar:

```
prioridad = heuristic(sucesor, problem)
```

por:

```
prioridad = nuevoCosto + heuristic(sucesor, problem)
```

La modificación se aceptó porque corresponde a la función de evaluación de A*. Conservé la estructura de uniformCostSearch y los nombres de las variables en español porque me ayudaban a entender la relación entre los dos algoritmos.

Cómo se verificó que funciona

Se comprobó que la función retornara una lista válida de acciones y alcanzara el estado objetivo. También se comparó A* usando nullHeuristic con búsqueda de costo uniforme.

manhattanHeuristic

Versión inicial propia reconstruida

```
def manhattanHeuristic(state, problem):
    if isinstance(state[0], (int, float)):
        return (
            abs(state[0] - problem.goal[0]) +
            abs(state[1] - problem.goal[1])
        )

    if len(state) == 2:
        position, hasModule = state

        # Error: las condiciones están invertidas
        if hasModule:
            target = problem.modulePosition
        else:
            target = problem.controlPosition

        return (
            abs(position[0] - target[0]) +
            abs(position[1] - target[1])
        )

    position, hasKit, pendingSystems = state

    if not hasKit:
        target = problem.kitPosition
    elif len(pendingSystems) > 0:
        target = min(
            pendingSystems,
            key=lambda system:
                abs(position[0] - system[0]) +
                abs(position[1] - system[1])
        )
    else:
        target = problem.controlPosition

    return (
        abs(position[0] - target[0]) +
        abs(position[1] - target[1])
    )
```

Prompts utilizados

- ¿Cómo se calcula la distancia Manhattan entre dos posiciones?
- ¿Cómo encuentro el sistema pendiente más cercano utilizando distancia Manhattan?

- ¿Cómo puedo evitar repetir la lógica de las fases?

Sugerencias relevantes recibidas

Se detectó que la condición `hasModule` estaba invertida. Si el agente todavía no tiene el módulo, debe dirigirse a `modulePosition`. Si ya lo tiene, debe dirigirse a `controlPosition`.

Cambios aceptados, rechazados o adaptados

Acepté cambiar la condición por:

```
if not hasModule:  
    return distance(position, problem.modulePosition)
```

```
    return distance(position, problem.controlPosition)
```

También acepté crear `_manhattanDistance` y `_phaseDistance`, porque esto evita repetir la lógica utilizada para interpretar el estado.

La versión final quedó resumida en:

```
def manhattanHeuristic(state, problem):  
    return _phaseDistance(state, problem, _manhattanDistance)
```

euclideanHeuristic

Versión inicial propia reconstruida

```
def euclideanHeuristic(state, problem):  
    def euclideanDistance(a, b):  
        # Error: fórmula incorrecta  
        return math.sqrt(  
            abs(a[0] - b[0]) +  
            abs(a[1] - b[1]))  
  
    if isinstance(state[0], (int, float)):  
        return euclideanDistance(state, problem.goal)  
  
    if len(state) == 2:  
        position, hasModule = state  
  
    if not hasModule:  
        return euclideanDistance(  
            position,  
            problem.modulePosition  
        )  
  
    return euclideanDistance(  
        position,  
        problem.controlPosition  
    )  
  
    position, hasKit, pendingSystems = state  
  
    if not hasKit:  
        return euclideanDistance(  
            position,
```

```

problem.kitPosition
)

if len(pendingSystems) > 0:
    return min(
        euclideanDistance(position, system)
        for system in pendingSystems
    )

return euclideanDistance(
    position,
    problem.controlPosition
)

```

Prompts utilizados

- ¿Cuál es la fórmula correcta de la distancia euclidiana?
- ¿Qué diferencia existe entre la distancia Manhattan y la euclidiana?
- ¿Puedo reutilizar la lógica de fases de la heurística Manhattan?
-

Sugerencias relevantes recibidas

Se señaló que la fórmula estaba equivocada. La distancia euclidiana debe calcularse elevando al cuadrado la diferencia de cada coordenada:

```
math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
```

También se sugirió reutilizar `_phaseDistance` para no duplicar las condiciones de cada fase.

Cambios aceptados, rechazados o adaptados

Acepté corregir la función auxiliar:

```

def _euclideanDistance(a, b):
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)

```

También reutilicé `_phaseDistance`, por lo que la función final quedó así:

```

def euclideanHeuristic(state, problem):
    return _phaseDistance(state, problem, _euclideanDistance)

```

systemRepairHeuristic

Versión inicial propia reconstruida

```

def systemRepairHeuristic(state, problem):
    position, hasKit, pendingSystems = state

    if len(pendingSystems) == 0:
        return _mazeDistance(
            position,
            problem.controlPosition,
            problem
        )

    if hasKit:
        # Problema: solo considera el sistema más cercano
        return min(
            _mazeDistance(position, system, problem)

```

```
for system in pendingSystems
)

# Problema: solo considera el recorrido hasta el kit
return _mazeDistance(
position,
problem.kitPosition,
problem
)
```

Prompts utilizados

- ¿Cómo construyo una heurística admisible si primero debo recoger un kit, reparar los sistemas y terminar en el control?
- ¿Por qué usar únicamente la distancia al sistema más cercano produce una heurística poco informada?
- ¿Puedo sumar las distancias de todos los sistemas pendientes?
- ¿Cómo puedo considerar las paredes del laberinto?
- ¿Cómo guardo las distancias para no calcularlas repetidamente?

Sugerencias relevantes recibidas

Se explicó que la versión inicial era muy poco informada porque solamente consideraba el sistema más cercano o el trayecto hasta el kit. Se recomendó utilizar distancias reales del laberinto mediante BFS y calcular una cota inferior con el máximo de:

```
distancia(posicion, sistema) + distancia(sistema, control)
```

También se sugirió almacenar las distancias calculadas en `problem.heuristicInfo`.

Cambios aceptados, rechazados o adaptados

Acepté utilizar `_mazeDistance` porque tiene en cuenta las paredes.